

- Q1. What is OOP?
- Q2. Why do we use OOP?
- Q3. What are the four Pillars of OOP?
- Q4. What is a class?
- Q5. What is an object?
- Q6. Difference between a class and an object.
- Q7. How do we create an object in C#?
- Q8. What is a constructor? Can a class have multiple constructors?
- Q9. Explain Different types of constructors
- Q10. What is a private constructor? And what are the real use cases of a private constructor?
- Q11. Can a class have multiple constructors?
- Q12. What are fields?
- Q13. What are properties?
- Q14. What are the available access modifiers?
- Q15. What is a static member vs. an instance member?

Encapsulation

- 1. What is Encapsulation?
- 2. Why is encapsulation important?
- 3. How do we achieve encapsulation in C#?
- 4. What's the difference between a field and a property in C#?
- 5. Why do we use public properties instead of public fields?
- 6. What are auto-implemented properties?
- 7. What is a readonly property?
- 8. What is a private set property?

9. How do access modifiers (private, public, protected, internal) support encapsulation?

10. Can a property have different access modifiers for get and set?

11. What are the differences between these two

```
public decimal Balance { get; set; }
```

```
public decimal Balance { get; private set; }
```

12. BankAccount class has a public Balance field that anyone can set directly, including negative values. How do you fix this using encapsulation?

13. Why might you make a setter private while keeping the getter public?

14. Predict/fix:

```
public class BankAccount
{
    public decimal Balance { get; private set; }
    public void Deposit(decimal amount)
    {
        if (amount > 0) Balance += amount;
    }
}
```

15. What does this property do, and why is it useful?

```
public class Person
{
    private string _name;
```

```
public string Name
{
    get => _name;

    set => _name = string.IsNullOrEmpty(value) ? throw new
ArgumentException("Invalid name") : value;
}
}
```